

Computation and Boundary Evidence for Scientific Software Repair

Rajarshi Ghoshal

rajarshi.ghoshal1@gmail.com

PhD applicant research task — Graz University of Technology

Abstract

Repair agents treat scientific repositories as flat text, searching for defects with `grep` and file reads. We pre-compute the computational structure — expressions, conditions, definitions, and library boundaries — and let the agent query it on demand. CBE (computation and boundary evidence) is a static extraction pipeline that indexes this structure without model calls. We test it against a matched baseline on 89 SWE-bench Science tasks with three replicate runs (534 verified attempts). Baseline solves 60; CBE solves 63. Aggregate near-parity masks a discipline-level split: physical-science tasks improve; life-science tasks decline. CBE uses 5.6% fewer output tokens but more input and work time. Pre-computed structure helps when the repair obligation is a calculation and misdirects when it is interface preservation.

1 Introduction

Task 103 in SWE-bench Science asks an agent to fix a composite-laminate stiffness calculation in `pyNastran`. The repair requires changing both the `ply` iteration and the accumulation formula. Expanding one without the other passes the public reproducer but fails the private tests. The patch is short; finding it means knowing which expression produces the returned matrix and what calls cross the `numpy` boundary.

A human engineer reading this code traces the computation: accumulation at line 1003, return at 1015, boundary call at 979. An ordinary repair agent reconstructs the same understanding from text search. CBE pre-computes and indexes this structure so the agent can query it directly.

CBE statically extracts computation graphs and boundary signatures before repair begins, then exposes them through three query endpoints alongside the agent’s normal shell tools. The baseline gets the same model, budget, tools, and containers — minus the evidence store.

We contribute: (1) a multi-language static extraction pipeline, (2) a targeted query interface for repair agents, and (3) a replicated evaluation that

separates discipline-level effects from aggregate noise.

CBE draws on traceability (Niu et al., 2023b,a, 2025; Pan et al., 2025), code property graphs (Yamaguchi et al., 2014), scientific model extraction (Pyarelal et al., 2020; ML4AI Team, 2024), and incremental parsing (Brunsfield et al., 2024). We evaluate on SWE-bench Science (Xu et al., 2026) with DeepSeek V4.1 Flash (DeepSeek, 2026); SWE-bench (Jimenez et al., 2024; Yang et al., 2024) and SciCode (Tian et al., 2024) are related benchmarks.

2 What CBE provides

The pipeline has three stages (Figure 2): trace, extract, query.

Trace. Run the public `reproduce.py` under an observer. The observer records which functions execute, guiding file selection for extraction. No model calls. Median duration: 70 s of the 1800 s budget.

Extract. Tree-sitter (Brunsfield et al., 2024) parses Python ASTs for scopes, assignments, binary expressions, return anchors, and branches. Joern handles C, C++, Fortran, and Cython where available; explicit fallback otherwise. External calls are labelled with their provider as interface assumptions, not correctness guarantees. Unresolved callees stay `unknown_external`.

Table 1 shows what CBE extracts for task 103 (`pyNastran`) — the opening example. The computation record links the accumulation at line 1003 to its return anchor. The boundary record identifies the `numpy` call and its scope.

Table 1. CBE evidence records for task 103 (pyNas-tran).

Record	Content
Computation	Scope, expressions, result anchors: accumulation at line 1003 $\rightarrow$ <code>return ABD</code> at 1015.
Condition	<code>len(z0) == len(z1)</code> at line 987.
Definition	Documented public behavior (not execution trace).
Boundary	<code>np.radians(thetad)</code> at line 979: provider <code>numpy</code> , caller-side scope.
Query interface (alongside ordinary shell)	
<code>find</code>	Keyword search over symbols, docstrings, and expressions.
<code>inspect</code>	Structured view: dependencies, conditions, boundaries.
<code>note</code>	Scratchpad for working hypotheses; never gates repair.

Query and repair. The agent accesses the three endpoints alongside its normal shell utilities. `find` searches the indexed structure; `inspect` retrieves computation scopes and boundary details; `note` records a working hypothesis without blocking repair. Inspection refreshes after each edit; the preparation trace stays fixed.

3 Protocol

We evaluate on 89 locked SWE-bench Science tasks across 20 disciplines (Xu et al., 2026), with 30 held for development. Each task runs three times per arm (534 verified attempts, zero gaps). Both arms share identical conditions (Table 2).

Table 2. Shared evaluation parameters.

Component	Detail
Model	DeepSeek V4.1 Flash (DeepSeek, 2026); temperature 0; <code>PYTHONHASHSEED=0</code> .
Replicates	89 tasks $\times$ 3 runs = 267 per arm; 534 verified.
Budget	1800 s, including preparation.
Verification	Separate container, 1800 s. Private tests not visible.
Network	None.
Baseline	Shell tools + public task material.
CBE	Same + evidence store + three query endpoints.

A solve requires the official verifier. Missing verdicts excluded, not failures. Paired bootstrap (20k resamples, fixed seed) and sign test excluding ties.

4 Results

Aggregate parity. Baseline solves 60 of 267 verified attempts; CBE solves 63 of 267 (Table 3). The confidence interval spans zero.

Table 3. Baseline solves 60, CBE solves 63. CBE generates 5.6% fewer output tokens.

Metric	Baseline	CBE
Scheduled attempts	267	267
Exact repairs / verdicts	60/267	63/267
Exact-repair rate	22.5%	23.6%
Private tests passed	2341/3219	2282/3219
Recorded input (M tokens)	952.4	1126.5
Recorded output (M tokens)	13.56	12.81
Mean recorded work (s)	1203.5	1271.4

Paired task-rate diff: +1.12 pp; 95% bootstrap $[-2.6, +4.9]$ pp; sign $p = 1.00$. All 534 attempts verified (267 per arm); mean work averages runtime across attempts with usage records.

Resource shift. 74 tasks tie, 8 favour CBE, 7 favour baseline. Output drops 5.6% (12.81M vs. 13.56M tokens); input rises 18.3%; work takes 68 s longer. The agent queries indexed structure instead of searching, but evidence packets increase context size.

Hidden partial changes. Even among exact ties, behaviour differs: 29 of 74 differ in private-test pass fractions (Table 4). CBE passes more tests on nine tasks; baseline passes more on 20.

Table 4. Exact-rate ties crossed with private-test pass fractions.

Exact rate	Private-test pass fractions		
	Baseline higher	Equal	CBE higher
Baseline higher	6	0	1
Equal	20	45	9
CBE higher	0	0	8

5 Where structure helps and where it hurts

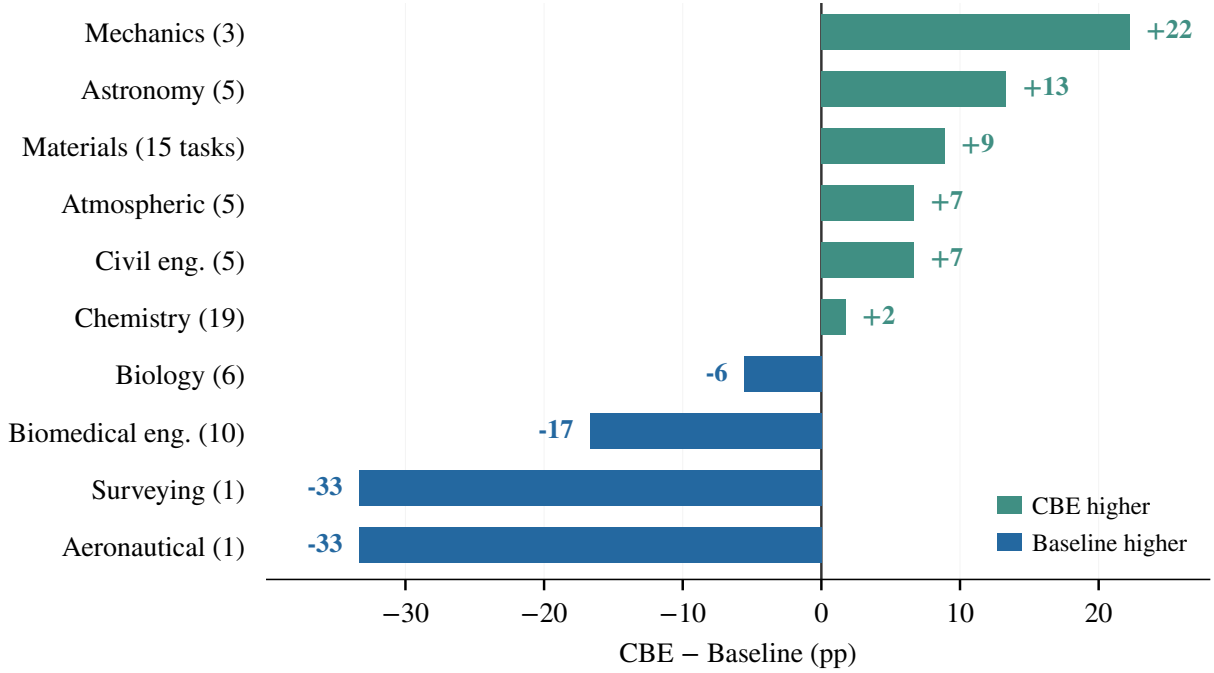


Figure 1. Solve-rate difference (CBE – Baseline) for the ten disciplines with non-zero change. Task counts in parentheses. Eight disciplines with identical solve rates across arms are omitted.

Figure 1 reveals a clean split beneath the aggregate parity. Six disciplines gain: mechanics (+22.2 pp, entirely task 103), astronomy (+13.3 pp, driven by sunpy), materials (+8.9 pp across DeePTB, deepmd-kit, and dpdata), atmospheric science (+6.7 pp), civil engineering (+6.7 pp), and chemistry (+1.8 pp). Four decline: biomedical engineering (−16.7 pp across four tasks including nilearn), biology (−5.6 pp across biopython and tskit), and the singletons surveying and aeronautical (−33.3 pp each). The remaining eight disciplines, including physics (7 tasks) and mathematics (5), show zero difference.

Connection to task paradigms. The benchmark organizes tasks into three paradigms (Xu et al., 2026). CBE marginally helps *Issue-driven* tasks (+1.5 pp), where localization matches the repair need; slightly hurts *Expert-exploratory* tasks (−1.8 pp), where indexed formulas anchor the agent instead of letting it reason about the anomaly; and has no effect on *Engineering-integration* tasks (0 pp), where multi-file architecture is the bottleneck. Of the benchmark’s four failure mechanisms, CBE addresses the first (knowledge deficit) for computation tasks and triggers the second (anchoring) on interface tasks. It does not touch system integration or scientific generalization.

Replication. Baseline and CBE never solve 60 and 59 tasks, respectively. Outcomes are mixed on 18 baseline and 19 CBE tasks; 11 tasks succeed in all three runs for each arm. About a fifth of tasks flip between runs; the confidence interval $([-2.6, +4.9] \text{ pp})$ spans zero.

Implications for representation design. The zero-change disciplines — physics (7), mathematics (5), and six singletons — contain bugs in test infrastructure and configuration, not mathematical pipelines. A representation that also indexes interface contracts and configuration schemas might extend the benefit without anchoring.

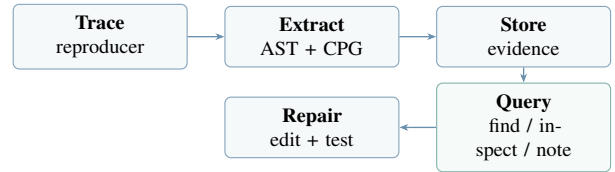


Figure 2. CBE pipeline: trace the reproducer, extract computation structure, store it, then the agent queries and repairs.

6 Case studies

Task / repository	Discipline	Baseline			CBE			Solved / verified
		Run 1	Run 2	Run 3	Run 1	Run 2	Run 3	
042 sunpy	Astronomy	15/16	15/16	6/16	16/16	7/16	16/16	0/3 → 2/3
074 DeePTB	Materials	9/10	9/10	2/10	10/10	10/10	9/10	0/3 → 2/3
103 pyNastran	Mechanics	2/4	2/4	2/4	4/4	4/4	2/4	0/3 → 2/3
067 xclim	Atmospheric	6/6	6/6	3/6	6/6	6/6	6/6	2/3 → 3/3
020 deepmd-kit	Materials	10/12	10/12	11/12	10/12	12/12	10/12	0/3 → 1/3
084 dpdata	Materials	5/7	5/7	7/7	7/7	5/7	7/7	1/3 → 2/3
101 PyBaMM	Chemistry	3/10	3/10	5/10	3/10	10/10	3/10	0/3 → 1/3
116 Pynite	Civil eng.	85/93	85/93	85/93	85/93	93/93	85/93	0/3 → 1/3
029 biopython	Biology	29/29	14/29	27/29	14/29	14/29	14/29	1/3 → 0/3
053 MONAI	Biomedical	2/6	6/6	2/6	5/6	5/6	2/6	1/3 → 0/3
087 dipy	Biomedical	5/5	0/5	1/5	0/5	2/5	2/5	1/3 → 0/3
093 MONAI	Biomedical	4/4	2/4	2/4	2/4	2/4	2/4	1/3 → 0/3
098 satpy	Surveying	11/11	8/11	11/11	8/11	11/11	8/11	2/3 → 1/3
056 poliastro	Aeronautical	10/10	10/10	10/10	10/10	10/10	1/10	3/3 → 2/3
022 nilearn	Biomedical	12/12	12/12	10/12	4/12	11/12	11/12	2/3 → 0/3

Cell: private tests passed / collected. Outlined + bold: official repair. Dash: no verdict.

Figure 3. Per-run private-test pass fractions for every task whose solve rate differs between arms. Outlined bold cells are official solves; bold task labels are the two cases below.

Table 5. Comparison of generated patches for divergent case studies.

Task	Baseline	CBE	Insight
103 pyNastran	Expand ply iteration; keep original B/D accumulation. All pass public.	Two fix both expansion and B/D; third keeps old accumulation.	Public check undertests. Full computation structure separates complete from partial repairs.
022 nilearn	Add label aggregation; preserve nearest-neighbour path.	Two alter argument handling (11/12 private); third fails (4/12).	Interface preservation is the obligation. CBE patches also alter existing arguments.

Task 103 (pyNastran). Figure 3 shows per-run outcomes. Baseline expands ply iteration but keeps original B/D accumulation; all pass public, all fail private. CBE exposes the accumulation formula; two of three attempts fix both expressions and pass.

Task 022 (nilearn). Baseline adds label aggregation while preserving the nearest-neighbour path. CBE attempts read coordinate-projection mathematics and alter argument handling — passing 11/12 private tests but failing backward compatibility.

7 Limitations and conclusion

CBE extracts syntax, not semantics: function scopes, calculation expressions, conditions, and call boundaries from ASTs and code property graphs. It does not model domain meaning, inter-module behavioural contracts, or backward-compatibility invariants, and unresolved callees are labelled `unknown_external` rather than inferred. The evaluation uses one model family and one scaffold, so the discipline split may not transfer to other agents. No component ablation separates

the contribution of the reproducer trace, the expression index, or the query endpoints. The measured effect is small: 15 of 89 tasks differ, the interval spans zero, and about a fifth of tasks flip between identical runs, which bounds what any single-run comparison can resolve. 81 of 89 tasks are pure Python, so the C, C++, Fortran, and Cython frontends are only lightly exercised.

The consistent direction of the split is the useful finding: pre-computed computation structure helps when the repair obligation is a calculation, and misdirects when it is preserving an interface. A representation that indexes interface contracts and configuration schemas alongside computation flows is the next step.

ChatGPT and Oh My Pi assisted with coding, debugging, analysis, and plotting. Code, receipts, and per-task outcomes: <https://github.com/rajarshighoshal/computation-boundary-evidence>.

References

- Max Brunsfeld et al. Tree-sitter: An incremental parsing system for programming tools, 2024. URL <https://tree-sitter.github.io>.
- DeepSeek. DeepSeek V4.1 Flash: Model card and API documentation, 2026. URL <https://api-docs.deepseek.com>.
- Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations*, 2024.
- ML4AI Team. GroMEt: A unified intermediate representation for scientific models and code. *DARPA ASKEM / SKEMA Consortium*, 2024.
- Feifei Niu, Wesley K. G. Assunção, Liguó Huang, Christoph Mayr-Dorn, Jidong Ge, Bin Luo, and Alexander Egyed. RAT: A refactoring-aware traceability model for bug localization. In *Proc. 45th International Conference on Software Engineering (ICSE)*, 2023a.
- Feifei Niu, Christoph Mayr-Dorn, Wesley K. G. Assunção, Liguó Huang, Jidong Ge, Bin Luo, and Alexander Egyed. The ABLoTS approach for bug localization: Is it replicable and generalizable? In *Proc. 20th International Conference on Mining Software Repositories (MSR)*, pages 152–162, 2023b.
- Feifei Niu, Liguó Huang, Christoph Mayr-Dorn, and Alexander Egyed. TVR: Automotive system requirement traceability validation and recovery through retrieval-augmented generation. *arXiv preprint arXiv:2504.15427*, 2025.
- Xinyi Pan, Feifei Niu, Liguó Huang, and Alexander Egyed. Requirements coverage-guided minimization for natural language test cases. *arXiv preprint arXiv:2505.20004*, 2025.
- Adarsh Pyarelal et al. AutoMATES: Automated model assembly from text, equations, and software. *arXiv preprint arXiv:2001.07295*, 2020.
- Minyang Tian, Luyu Gao, Shizhuo Dylan Zhang, et al. SciCode: A research coding benchmark curated by scientists. *arXiv preprint arXiv:2407.13168*, 2024.
- Zhipeng Xu, Jiahao Lu, Yining Zheng, Yuxin Wang, and Xipeng Qiu. SWE-bench Science: Can coding agents resolve engineering tasks in science? *arXiv preprint arXiv:2608.19799v2*, 2026.
- Fabian Yamaguchi, Nico Golde, Daniel Arp, and Konrad Rieck. Modeling and discovering vulnerabilities with code property graphs. In *IEEE Symposium on Security and Privacy*, 2014. doi: 10.1109/SP.2014.44.
- John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-bench Verified: A stricter subset for real-world software engineering. *arXiv preprint arXiv:2407.01489*, 2024.